



8장. 텍스트 분석

2조 장서연 고은비 백재은

목차

A파트 – 8-0, 8-1장

B파트 – 8-2장

C파트 – 8-3장

A파트 – 8-5장



A파트 – 8-0, 8-1장



8-0 텍스트 분석 개요

텍스트 분석

- 비정형 데이터인 텍스트를 분석하는 것

NLP vs 텍스트 분석

NLP (National Language Processing)

- 머신이 인간의 언어를 이해/해석하는 데 중점
- 예) 기계 번역, 질의 응답 챗봇

텍스트 분석 (Text Analytics, Text Mining)

- 비정형 텍스트에서 의미 있는 정보 추출
- 머신러닝, 언어 이해, 통계 등을 활용해 모델을 수립하고 정보를 추출해
비즈니스 인텔리전스/예측 분석 등 분석 작업 수행

8-0 텍스트 분석 개요

텍스트 분석 기술 영역

텍스트 분류 (Text Classification/Categorization)

- 문서가 특정 분류/카테고리에 속하는 것을 예측
- 지도 학습

감성 분석 (Sentiment Analysis)

- 텍스트에서 나타나는 감정/판단/의견/기분 등의 주관적 요소 분석
- 지도학습/비지도 학습

텍스트 요약 (Summarization)

- 텍스트 내에서 중요한 주제나 중심 사상 추출
- Topic Modeling

텍스트 군집화(Clustering)와 유사도 측정

- 비슷한 유형의 문서에 대해 군집화 수행 / 문서들간의 유사도 측정, 비슷한 문서끼리 모음
- 텍스트 분류의 비지도 학습 수행 방법

8-0 텍스트 분석 개요

텍스트 분석 기술 활용 분야

1.4.2 활용 분야

실무	연구
스팸 필터링	사회동향 분석
이슈 검출/트래킹	이슈 트래킹
정보 검색	온라인 행동 분석
자살률 예측	연구분야 탐색
주가 예측	질병관계 예측
소비자 인식 조사	정책전략 수립
경쟁사 분석	

8-1 텍스트 분석의 이해

피처 벡터화/추출 (Feature Vectorization / Feature Extraction)

- 텍스트를 머신러닝에 적용하기 위해서는 비정형 텍스트 데이터의 **피처 추출 방법**과 **추출된 피처에 의미있는 값**을 부여하는지가 중요
 - 텍스트를 **word 기반** 다수의 피처로 추출,
 - 피처에 **단어 빈도수와 같은 숫자 값** 부여
- 텍스트는 단어의 조합인 벡터값으로 표현
- BOW(Bag of Words), Word2Vec 등

8-1 텍스트 분석의 이해

텍스트 분석 수행 과정

Step ① 데이터 준비

- API 호출, 웹 크롤링

Step ② 데이터 전처리

- 데이터 정규화 : 표현 방법이 다른 단어들 통합 예) 감사하니당 -> 감사합니다
- 데이터 분리 : 데이터를 특성에 따라 분리할 필요가 있을 때 분리
- 형태소 분석(토큰화) : 품사태깅, 일정한 의미가 있는 가장 작은 말의 단위(형태소) 변환
- 개체명 인식 : 이름을 가진 개체로 인식 예) 소희 - 사람, 동국대 - 조직
- 원형 복원 : 형용사/동사 > 원형, 어간/어미 > 표현형 예) 쉬고싶다 > 쉬다, 보다/보니 > 보-
- 불용어 제거 : 조사,접미사 예) 나,너,은,는,이,가,하다 등
- 단어 빈도 분석 : 불용어 및 빈출어의 제거 여부/필요한 단어들의 올바른 추출 확인

8-1 텍스트 분석의 이해

텍스트 분석 수행 과정

Step ③ 데이터 분석

- 동시 출현 분석
- 문서요약
- 텍스트 생성
- 키워드 추출
- 군집화
- 네트워크 분석
- 단어 임베딩
- 감성 분석
- 토픽 분석

Step ④ 분석 결과 시각화

- 히스토그램
- 네트워크 다이어그램
- 덴드로그램
- 테이블
- 워드 클라우드
- 히트맵

Step ⑤ 보고

- 분석 결과 비교/해석, 보고

8-1 텍스트 분석의 이해

파이썬 기반 NLP, 텍스트 분석 패키지

📌 NLTK(Natural Language Toolkit for Python)



- 가장 기본적인 NLP 패키지
- 방대한 데이터셋, 서브 모듈
- 수행 속도 아쉬움, 대량 데이터 제대로 활용 ❌, 실무보다는 교육 및 연구에 사용
- 주요 기능 : 말뭉치, 토큰 생성, 형태소 분석, 품사 태깅

📌 Genism



- 토픽 모델링에서 두각을 나타내는 패키지
- 파이썬에서 제공하는 Word2Vec 라이브러리
- 효율성 + 확장 가능성으로 인해 폭넓게 사용됨
- 주요기능 : 임베딩(Word2Vec), 토픽 모델링, LDA(Latent Dirichlet Allocation)

8-1 텍스트 분석의 이해

파이썬 기반 NLP, 텍스트 분석 패키지



- 뛰어난 수행 성능으로 주목받는 NLP 패키지
- 생산용 소프트웨어 제공에 중점, 딥러닝 워크플로우 지원
- 주요 기능 : 토큰화, POS 태깅, SBD(Sentence Boundary Detection)



- 한국어 처리를 위한 파이썬 라이브러리
- 파이썬에서 활용 가능한 오픈 소스 형태소 분석기
 - 기존에 공개된 여러 분석기를 한번에 설치 + 동일한 방법으로 사용 가능하게 해줌
- 주요기능 : 형태소 분석(morphs()), 품사태깅(pos())

B파트 – 8-2장 텍스트 정규화



텍스트 정규화

- 클렌징 (Cleansing)
- 토큰화 (tokenization)
- 스톱 워드 제거
- Stemming, Lemmatization

클렌징 (Cleaning)

HTML DOCTYPE
declaration

`<!DOCTYPE html>`

`<html lang="en">`

html
element

head
element

`<head>`

`<meta charset="utf-8">`

`<meta http-equiv="X-UA-Compatible" content="IE=edge">`

`<title>R: The R Project for Statistical Computing</title>`

`</head>`

body
element

`<body>`

`<p><a href="/Rlogo.png"/></a></p>`

`<p><small><a href="/Rlogo.png"/>[Home]</a></small></p>`

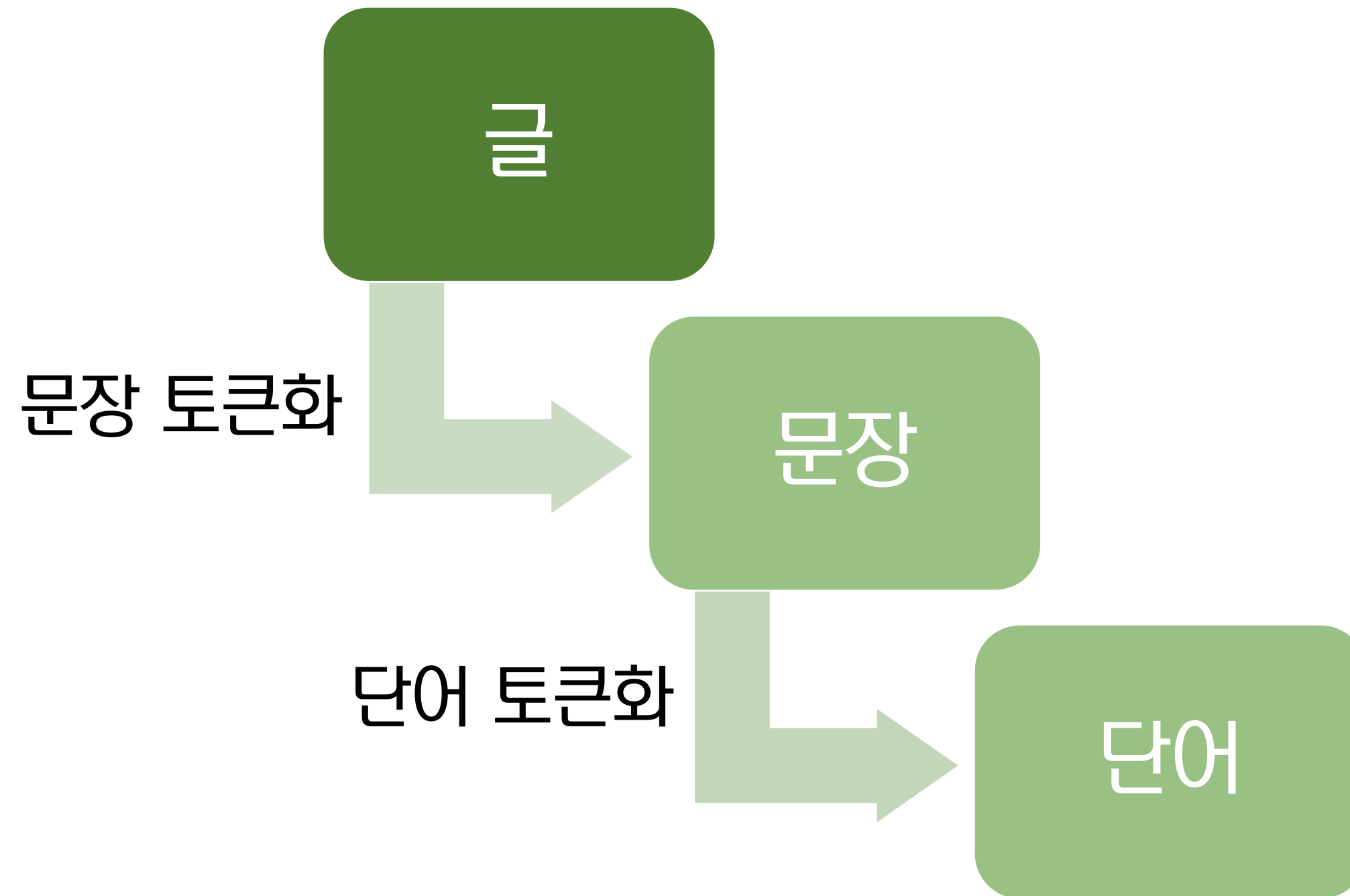
`<h2 id="download">Download</h2>`

`...`

`</body>`

`</html>`

토큰화 (Tokenization)



문장 토큰화

```
from nltk import sent_tokenize
import nltk

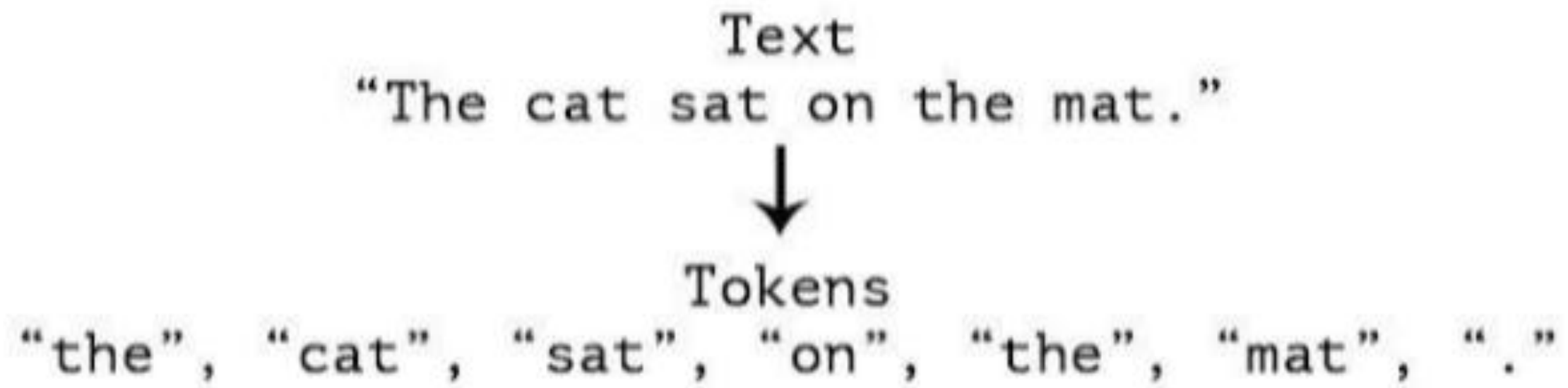
nltk.download('punkt_tab')

text_sample = 'The Matrix is everywhere its all around us, here even in this room. \
    You can see it out your window or on your television. \
    You feel it when you go to work, or go to church or pay your taxes.'
sentences = sent_tokenize(text=text_sample)
print(type(sentences), len(sentences))
print(sentences)
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]   C:\Users\SAMSUNG\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
```

```
<class 'list'> 3
['The Matrix is everywhere its all around us, here even in this room.', 'You can see it out
your window or on your television.', 'You feel it when you go to work, or go to church or p
ay your taxes.']
```


단어 토큰화



단어 토큰화

```
[25]: from nltk import word_tokenize
sentence = "The Matrix is everywhere its all around us, here even in this room."
words = word_tokenize(sentence)
print(type(words), len(words))
print(words)

<class 'list'> 15
['The', 'Matrix', 'is', 'everywhere', 'its', 'all', 'around', 'us', ',', 'here', 'even', 'in', 'this', 'room', '.']
```



단어 토큰화

```
from nltk import word_tokenize , sent_tokenize

def tokenize_text(text):
    sentences =sent_tokenize(text)
    word_tokens=[word_tokenize(sentence) for sentence in sentences]
    return word_tokens

word_tokens=tokenize_text(text_sample)
print(type(word_tokens), len(word_tokens))
print(word_tokens)
```

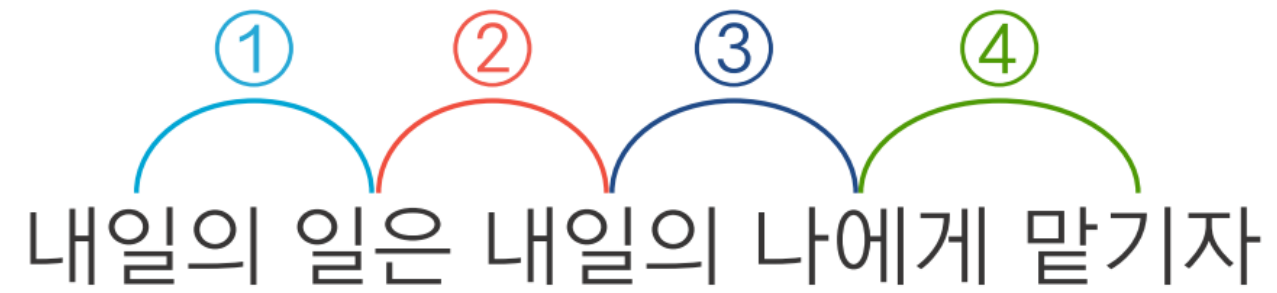
```
<class 'list'> 3
```

```
[['The', 'Matrix', 'is', 'everywhere', 'its', 'all', 'around', 'us', ',', 'here', 'even',  
'in', 'this', 'room', '.'], ['You', 'can', 'see', 'it', 'out', 'your', 'window', 'or', 'o  
n', 'your', 'television', '.'], ['You', 'feel', 'it', 'when', 'you', 'go', 'to', 'work',  
,', 'or', 'go', 'to', 'church', 'or', 'pay', 'your', 'taxes', '.']]]
```

단어 토큰화 - ngram

원문

①
②
③
④
내일의 일은 내일의 나에게 맡기자



바이그램

- ① 내일의 일은
② 일은 내일의
③ 내일의 나에게
④ 나에게 맡기자

스톱 워드 (Stop Word) 제거

emissions
report action united that
at paris is change have would it
global ice be from greenhouse this average
countries more all summit
about by and the convention nations
to for gas with co2 as a of climate
agreement rise warming c earth on
in sea are wollen level
parties

스톱 워드 (Stop Word) 제거

```
print('영어 stop words 개수:', len(nltk.corpus.stopwords.words('english')))  
print(nltk.corpus.stopwords.words('english')[:20])
```

영어 stop words 개수: 198

['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'an
v', 'are', 'aren', "aren't", 'as', 'at', 'be', 'because', 'been']

스톱 워드 (Stop Word) 제거

```
import nltk

stopwords=nltk.corpus.stopwords.words('english')
all_tokens = []

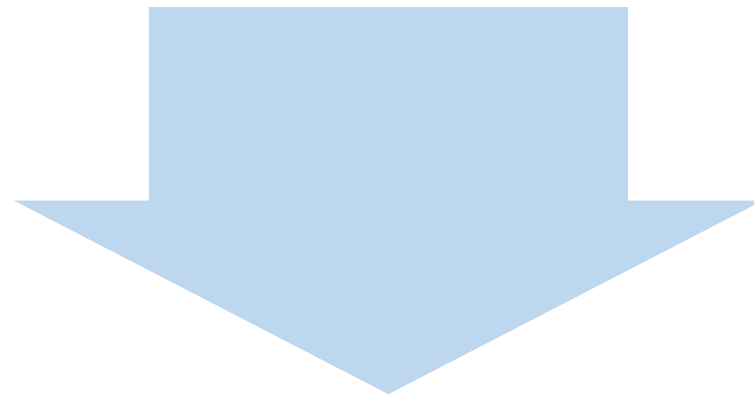
for sentence in word_tokens:
    filtered_words= []
    for word in sentence:
        word =word.lower() # 소문자로 변환
        if word not in stopwords: # stop word가 아니면 추가
            filtered_words.append(word)
    all_tokens.append(filtered_words)

print(all_tokens)
```

```
[['matrix', 'everywhere', 'around', 'us', ',', 'even', 'room', '.'], ['see', 'window', 'television', '.'], ['feel', 'go', 'work', ',', 'go', 'church', 'pay', 'taxes', '.']]
```

스톱 워드 (Stop Word) 제거

```
[[ 'The', 'Matrix', 'is', 'everywhere', 'its', 'all', 'around', 'us', ',', 'here', 'even',  
'in', 'this', 'room', '.'], ['You', 'can', 'see', 'it', 'out', 'your', 'window', 'or', 'o  
n', 'your', 'television', '.'], ['You', 'feel', 'it', 'when', 'you', 'go', 'to', 'work',  
,', 'or', 'go', 'to', 'church', 'or', 'pay', 'your', 'taxes', '.']]
```



```
[[ 'matrix', 'everywhere', 'around', 'us', ',', 'even', 'room', '.'], ['see', 'window', 'tel  
evision', '.'], ['feel', 'go', 'work', ',', 'go', 'church', 'pay', 'taxes', '.']]
```


Stemming vs Lemmatization

Stemming

adjustable → adjust|
formality → formalit|
formality → form|al
airliner → airlin|

Lemmatization

was → (to) be
better → good
meeting → meeting

Stemming

```
from nltk.stem import LancasterStemmer
stemmer = LancasterStemmer()

print(stemmer.stem('working'), stemmer.stem('works'), stemmer.stem('worked'))
print(stemmer.stem('amusing'), stemmer.stem('amuses'), stemmer.stem('amused'))
print(stemmer.stem('happier'), stemmer.stem('happiest'))
print(stemmer.stem('fancier'), stemmer.stem('fanciest'))
```

work work work

amus amus amus

happy happiest

fant fanciest

Lemmatization

```
from nltk.stem import WordNetLemmatizer
import nltk
nltk.download('wordnet')

lemma = WordNetLemmatizer()
print(lemma.lemmatize('amusing', 'v'), lemma.lemmatize('amuses', 'v'), lemma.lemmatize('amuse', 'v'))
print(lemma.lemmatize('happier', 'a'), lemma.lemmatize('happiest', 'a'))
print(lemma.lemmatize('fancier', 'a'), lemma.lemmatize('fanciest', 'a'))
```

```
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\SAMSUNG\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

amuse amuse amuse

happy happy

fancy fancy

C파트 – 8-3장 Bag of Words



Contents

- Intro
- BOW 피처 벡터화
- 사이킷런의 Count 및 TF-IDF 벡터화 구현
- BOW 벡터화를 위한 희소 행렬
- 희소 행렬 -C00 형식
- 희소 행렬 - CSR 형식

Intro

Bag of Words 모델이란?

-> 문서가 가지는 모든 단어를 문맥이나 순서를 무시하고 일괄적으로 단어에 대해 빈도 값을 부여하여 피쳐 값을 추출하는 모델

Like 롯데리아 양념감자...

피쳐 추출 방식 예시

S1: My wife likes to watch baseball games and my daughter likes to watch baseball games too.

S2: My wife likes to play baseball.

Intro

단어 중복 제거 -> 칼럼 형태 나열 -> 인덱스 부여

'and':0, 'baseball':1, 'daughter':2, 'games':3, 'likes':4, 'my':5, 'play':6, 'to':7, 'too':8,
'watch':9, 'wife':10

단어가 나타나는 횟수 기재

	Index 0	Index 1	Index 2	Index 3	Index 4	Index 5	Index 6	Index 7	Index 8	Index 9	Index 10
	and	baseball	daughter	games	likes	my	play	to	too	watch	wife
문장 1	1	2	1	2	2	2		2	1	2	1
문장 2		1			1	1	1	1			1

→ 문장 1에서 baseball은 2회 나타남

Intro

장점

- 쉽고 빠른 구축
- 문서의 특징을 잘 나타낼 수 있는 모델

단점

- 문맥 의미 반영 부족: 보완하기 위해 n_gram 기법 활용
- 희소 행렬 문제: 대규모의 칼럼으로 구성된 행렬에서 대부분의 값이 0으로 채워지는
행렬 발생

BOW 피처 벡터화

피처 벡터화

- 텍스트를 특정 의미를 가지는 숫자형 값인 벡터 값으로 변환하는 것
- Why? 머신러닝 알고리즘은 일반적으로 숫자형 피처를 데이터로 입력받아 동작하기 때문에 텍스트와 같은 데이터는 머신러닝 알고리즘에 바로 입력 불가
- 각 문서의 텍스트를 단어로 추출 -> 피처 할당 -> 단어의 발생 빈도와 같은 값을 피처에 값으로 부여 -> 각 문서를 단어 피처의 발생 빈도 값으로 구성된 벡터로 만드는 기법
- 넓은 범위의 피처 추출...

결국...

피처 벡터화 수행 => 모든 문서에서 모든 단어를 칼럼 형태로 나열, 각 문서에서 해당 단어의 횟수나 정규화된 빈도를 값으로 부여하는 데이터 세트 모델로 변경하는 것

BOW 피쳐 벡터화



〈 BOW 피쳐 벡터화 〉

BOW 피처 벡터화

카운트 벡터화

- 단어 피터에 값을 부여할 때 각 문서에 해당 단어가 나타나는 횟수(Count)를 부여하는 경우
- 카운트 up = 중요도 up
- But... 언어 특성상 문장에서 자주 사용되는 단어에 높은 값 부여

TF-IDF 벡터화

- 개별 문서에서 자주 나타나는 단어 높은 가중치, 모든 문서에서 전반적으로 자주 나타나는 단어에 대해 패널티를 주는 방식으로 값 부여
- 분쟁, 종교 대립, 유혈 사태 vs 많은, 빈번하게, 당연히, 조직, 업무
- 텍스트가 길고 문서의 개수가 많은 경우 더 좋은 예측 성능 보장!

BOW 피처 벡터화



한 개의 문서(Document)



모든 문서들(Corpus)

Term Frequency

The	Matrix	is	nothing	but	an	advertising	gimmick
40	5	50	12	20	45	3	2

Document Frequency

The	Matrix	is	nothing	but	an	advertising	gimmick
2000	190	2300	500	1200	3000	52	12

$$TFIDF_i = TF_i * \log \frac{N}{DF_i}$$

TF_i = 개별 문서에서의 단어 i 빈도

DF_i = 단어 i 를 가지고 있는 문서 개수

N = 전체 문서 개수

사이킷런의 Count 및 TF-IDF 벡터화 구현

CountVectorizer

- 카운트 기반의 벡터화 구현 클래스
- 소문자 일괄 변환, 토큰화, 스톱 워드 필터링 등의 텍스트 전처리 함께 수행
- 텍스트 전처리 및 피처 벡터화를 위한 입력 파라미터 설정하여 동작
- `fit( )` & `transform( )` 을 통해 피처 벡터화된 객체 반환

사이킷런의 Count 및 TF-IDF 벡터화 구현

파라미터 명	파라미터 설명
max_df	전체 문서에 걸쳐서 너무 높은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터
min_df	전체 문서에 걸쳐서 너무 낮은 빈도수를 가지는 단어 피처를 제외하기 위한 파라미터
max_features	추출하는 피처의 개수를 제한하며 정수로 값을 지정
stop_wors	english'로 지정 시 영어의 스톱 워드로 지정된 단어는 추출에서 제외
n_gram_range	BoW 모델의 단어 순수를 어느 정도 보강하기 위한 n_gram 범위 설정, 튜플 형태
analyzer	피처 추출을 수행한 단위 지정, 디폴트는 word
token_pattern	토큰화를 수행하는 정규 표현식 패턴 지정, 디폴트 값은 <code>\b\w\w + \w</code>
Tokenizer	토큰화를 별도의 커스텀 함수로 이용 시 적용

사이킷런의 Count 및 TF-IDF 벡터화 구현



TF-IDF 벡터화 -> TfidfVectorizer 클래스 이용
파라미터와 변환 방법은 CounterVectorizer와 동일!

BOW 벡터화를 위한 희소 행렬

문서의 단어 추출 -> 피처로 벡터화 -> 피처 칼럼 생성
-> 중복 제거 -> 피처 생성 -> 수만 ~ 수십만 단어...

If ... n-gram 증가 시 칼럼 수 up

But... 대규모 행렬 생성되더라도 레코드 각 문서의 단어 수는 제한적

-> 값은 0! => **희소행렬**

BOW 벡터화를 위한 희소 행렬

← 수십만 개의 칼럼 →

	단어 1	단어 2	단어 3		단어 1000		단어 2000		단어 10000		단어 20000		단어 100000
수천~ 수만 개 레코드	문서1	1	2	2	0	0	0	0	0	0	0	0	0
	문서2	0	0	1	0	0	1	0	0	1	0	0	1
	문서 ...												
	문서 10000	0	1	3	0	0	0	0	0	0	0	0	0

BOW의 Vectorization 모델은 너무 많은 0값이 메모리 공간에 할당되어 많은 메모리 공간이 필요하며
연산 시에도 데이터 액세스를 위한 많은 시간이 소모됩니다.

희소 행렬 - C00 형식

C00 형식 coordinate

- 0이 아닌 데이터만 별도의 데이터 배열에 저장, 데이터가 가리키는 행과 열의 위치를 별도의 배열로 저장하는 방식
- 예시...
- $\begin{bmatrix} 3 & 1 & 2 \end{bmatrix}$
 $\begin{bmatrix} 0 & 2 & 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 3 & 1 & 2 \end{bmatrix} \Rightarrow (\text{row}, \text{col}) \Rightarrow (0,0), (0,2), (1, 1) \Rightarrow [0, 0, 1], [0, 2, 1]$
- 희소 행렬 변환 시 사이파이 이용, sparse 패키지 내 모듈

희소 행렬 - C00 형식

✓
0초

```
[2] import numpy as np  
  
dense = np.array([ [3, 0, 1], [0, 2, 0] ])
```

[[3, 1, 2], [0, 2, 0]]
넘파이 ndarray 객체 생성

✓
0초

```
[3] from scipy import sparse  
  
data = np.array([3, 1, 2])  
  
row_pos = np.array([0, 0, 1])  
col_pos = np.array([0, 2, 1])  
  
sparse_coo = sparse.coo_matrix((data, (row_pos, col_pos)))
```

사이파이 coo_matrix 클래스 이용
C00 형식 희소 행렬 변환
0 아닌 데이터 추출
행 위치와 열 위치 각각 배열 생성
Sparse 패키지의 coo_matrix 이용
coo 형식으로 희소 행렬 생성

-> sparse_coo: C00 형식의 희소 행렬
객체 변수

희소 행렬 - C00 형식

✓
0초

```
[4] sparse_coo.toarray()
```

```
→ array([[3, 0, 1],  
         [0, 2, 0]])
```

toarray() 메서드 이용, 다시 밀집 형태
행렬로 출력
→ 원래 데이터 행렬로 출력

희소 행렬 - CSR 형식

CSR 형식 Compressed Sparse Row

- COO 형식이 행렬의 위치를 나타내기 위해 반복적인 위치 데이터를 사용해야 하는 문제점을 해결한 방식

COO의 문제점

행 위치 배열이 0부터 순차적으로 증가하는 값!
-> 시작 위치만 표기하면 되지 않을까~?

Dense 형식의 원본 데이터

[[0,0,1,0,0,5],
[1,4,0,3,2,5],
[0,6,0,3,0,0],
[2,0,0,0,0,0],
[0,0,0,7,0,8],
[1,0,0,0,0,0]]

0이 아닌 데이터 값 배열

[1, 5, 1, 4, 3, 2, 5, 6, 3, 2, 7, 8, 1]

0이 아닌 데이터 값의 행과 열 위치

(0, 2), (0, 5)
(1, 0), (1, 1), (1, 3), (1, 4), (1, 5)
(2, 1), (2, 3)
(3, 0)
(4, 3), (4, 5)
(5, 0)

행 위치 배열

[0, 0, 1, 1, 1, 1, 1, 2, 2, 3, 4, 4, 5]

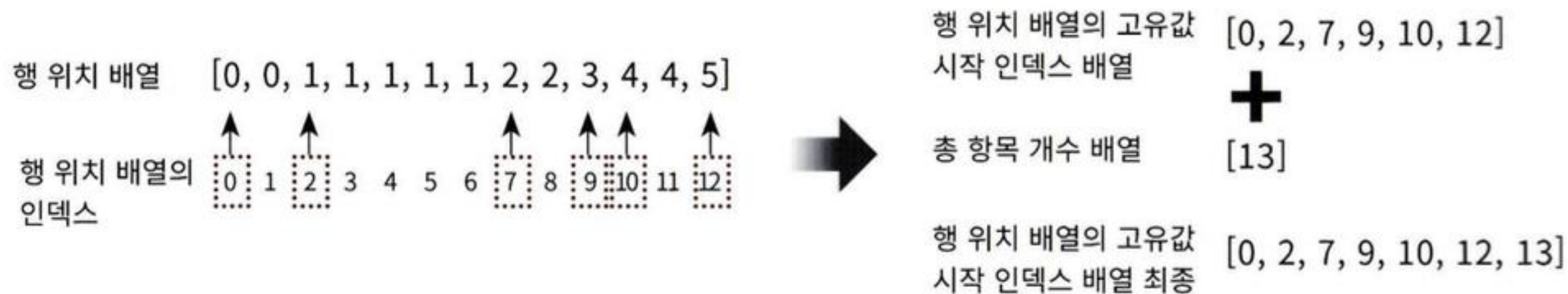
열 위치 배열

[2, 5, 0, 1, 3, 4, 5, 1, 3, 0, 3, 5, 0]

희소 행렬 - CSR 형식

CSR 형식 Compressed Sparse Row

- COO 형식이 행렬의 위치를 나타내기 위해 반복적인 위치 데이터를 사용해야 하는 문제점을 해결한 방식
- 행 위치 배열이 0부터 순차적으로 증가하는 값! -> 시작 위치만 표기하면 되지 않을까~?



- 사이파이 `csr_matrix` 클래스
- 생성 파라미터: 0이 아닌 데이터 배열, 열 위치 배열, 행 위치 배열의 고유한 값의 시작 위치 배열

희소 행렬 - CSR 형식

```
0초
▶ from scipy import sparse
import numpy as np

dense2 = np.array([[0, 0, 1, 0, 0, 5],
                  [1, 4, 0, 3, 2, 5],
                  [0, 6, 0, 3, 0, 0],
                  [2, 0, 0, 0, 0, 0],
                  [0, 0, 0, 7, 0, 8],
                  [1, 0, 0, 0, 0, 0]])

# 0이 아닌 데이터 추출
data2 = np.array([1, 5, 1, 4, 3, 2, 5, 6, 3, 2, 7, 8, 1])

# 행 위치와 열 위치를 각각 array로 생성
row_pos = np.array([0, 0, 1, 1, 1, 1, 1, 2, 2, 3, 4, 4, 5])
col_pos = np.array([2, 5, 0, 1, 3, 4, 5, 1, 3, 0, 3, 5, 0])

# COO 형식으로 변환
sparse_coo = sparse.coo_matrix((data2, (row_pos, col_pos)))

# 행 위치 배열의 고유값 값의 시작 위치 인덱스를 배열로 생성
row_pos_ind = np.array([0, 2, 7, 9, 10, 12, 13])

# CSR 형식으로 변환
sparse_csr = sparse.csr_matrix((data2, col_pos, row_pos_ind))

print('COO 변환된 데이터가 제대로 되었는지 다시 Dense로 출력 확인')
print(sparse_coo.toarray())
print('CSR 변환된 데이터가 제대로 되었는지 다시 Dense로 출력 확인')
print(sparse_csr.toarray())
```

```
⇒ COO 변환된 데이터가 제대로 되었는지 다시 Dense로 출력 확인
[[0 0 1 0 0 5]
 [1 4 0 3 2 5]
 [0 6 0 3 0 0]
 [2 0 0 0 0 0]
 [0 0 0 7 0 8]
 [1 0 0 0 0 0]]
CSR 변환된 데이터가 제대로 되었는지 다시 Dense로 출력 확인
[[0 0 1 0 0 5]
 [1 4 0 3 2 5]
 [0 6 0 3 0 0]
 [2 0 0 0 0 0]
 [0 0 0 7 0 8]
 [1 0 0 0 0 0]]
```

실제 사용 시엔...

✓
0초

```
[8] dense3 = np.array([[0, 0, 1, 0, 0, 5],  
                        [1, 4, 0, 3, 2, 5],  
                        [0, 6, 0, 3, 0, 0],  
                        [2, 0, 0, 0, 0, 0],  
                        [0, 0, 0, 7, 0, 8],  
                        [1, 0, 0, 0, 0, 0]])
```

```
coo = sparse.coo_matrix(dense3)  
csr = sparse.csr_matrix(dense3)
```


A파트 – 8-5장



8-5 감성 분석

감성 분석

- 문장의 주관적인 감성/의견/감정/기분 파악하는 방법
- 문장 내 텍스트가 나타내는 주관적 단어/문맥 기반으로 감성수치 계산
- 감성 지수 > 긍정 감성 지수 + 부정 감성 지수

지도 학습 기반 감성 분석

- 학습데이터, 타깃 레이블 값 기반으로 학습 수행, 다른 데이터 감성 분석 예측

비지도 학습 기반 감성 분석

- ‘Lexicon’ 이라는 감성 어휘 사전 이용
- Lexicon의 감성 분석을 위한 용어와 문맥에 대한 다양한 정보를 기반으로 긍정/부정 감성 여부 판단

8-5 감성 분석 실습 – 지도 학습

데이터 로드

```
import pandas as pd

review_df = pd.read_csv('/content/drive/MyDrive/Sample_data/labeledTrainData.tsv', header=0, sep="\t", quoting=3)
review_df.head(3)
```

	id	sentiment	review
0	"5814_8"	1	"With all this stuff going down at the moment ...
1	"2381_9"	1	"\"The Classic War of the Worlds\" by Timothy ...
2	"7759_3"	0	"The film starts with a manager (Nicholas Bell...



Bag of Words Meets Bags of Popcorn / IMDB 영화평 – tsv(tab separated values)

- Id : 각 데이터의 ID
- Sentiment : 타겟 레이블, 0: 부정/ 1: 긍정
- review : 영화평의 텍스트

8-5 감성 분석 실습 – 지도 학습

 태그 및 숫자/특수문자 공백으로 변경

```
print(review_df['review'][0])
```

of drugs are bad m'kay.
 Visually impressive but of course this is all about Michael Jackson so unless you remo

```
import re
```

```
# <br> html 태그는 replace 함수로 공백으로 변환
```

```
review_df['review'] = review_df['review'].str.replace('<br />', ' ')
```

```
# 파이썬의 정규 표현식 모듈인 re를 이용하여 영어 문자열이 아닌 문자는 모두 공백으로 변환
```

```
review_df['review'] = review_df['review'].apply( lambda x : re.sub("[^a-zA-Z]", " ", x) )
```

-
 태그 : html 형식에서 추출된 흔적,
.str.replace()로 제거
- 파이썬의 re 모듈 : 정규 표현식 지원
 - [^a-zA-Z] : 영어 대소문자가 아닌 모든 문자 찾기
 - re.sub(정규표현식, 바꿀 문자열, 대상문자열, 바꿀횟수(default=0,다)
: 문자열에서 정규 표현식 패턴을 찾아 다른 문자열로 변환

8-5 감성 분석 실습 – 지도 학습

X, y 설정 / 데이터 분리

```
from sklearn.model_selection import train_test_split

class_df = review_df['sentiment']
feature_df = review_df.drop(['id', 'sentiment'], axis=1, inplace=False)

X_train, X_test, y_train, y_test = train_test_split(feature_df, class_df, test_size=0.3, random_state=156)

X_train.shape, X_test.shape

((17500, 1), (7500, 1))
```

- X: review
- y(결정값 클래스) : sentiment(0,1) > 이진 분류
- train_test_split()
학습 데이터 : 17500개, 테스트 데이터 : 7500

8-5 감성 분석 실습 – 지도 학습

CounterVectorizer로 피쳐 벡터화, 분류 알고리즘 학습

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score

# 스톱 워드는 English, filtering, ngram은 (1,2)로 설정해 CountVectorization수행.
# LogisticRegression의 C는 10으로 설정.
pipeline = Pipeline([
    ('cnt_vect', CountVectorizer(stop_words='english', ngram_range=(1,2) )),
    ('lr_clf', LogisticRegression(solver='liblinear', C=10))]

# Pipeline 객체를 이용하여 fit(), predict()로 학습/예측 수행. predict_proba()는 roc_auc때문에 수행.
pipeline.fit(X_train['review'], y_train)
pred = pipeline.predict(X_test['review'])
pred_probs = pipeline.predict_proba(X_test['review'])[:,1]

print('예측 정확도는 {0:.4f}, ROC-AUC는 {1:.4f}'.format(accuracy_score(y_test, pred),
                                                    roc_auc_score(y_test, pred_probs)))
```

예측 정확도는 0.8861, ROC-AUC는 0.9503

8-5 감성 분석 실습 – 지도 학습

TF-IDF로 피처 벡터화, 분류 알고리즘 학습

```
# 스톱 워드는 english, filtering, ngram은 (1,2)로 설정해 TF-IDF 벡터화 수행.  
# LogisticRegression의 C는 10으로 설정.  
pipeline = Pipeline([  
    ('tfidf_vect', TfidfVectorizer(stop_words='english', ngram_range=(1,2) )),  
    ('lr_clf', LogisticRegression(solver='liblinear', C=10))])  
  
pipeline.fit(X_train['review'], y_train)  
pred = pipeline.predict(X_test['review'])  
pred_probs = pipeline.predict_proba(X_test['review'])[:,1]  
  
print('예측 정확도는 {0:.4f}, ROC-AUC는 {1:.4f}'.format(accuracy_score(y_test, pred),  
                                                         roc_auc_score(y_test, pred_probs)))
```

예측 정확도는 0.8936, ROC-AUC는 0.9598

8-5 감성 분석

비지도 학습 기반 감성 분석

- Lexicon 기반
- Y (결정된 레이블값) ✕

Lexicon

- 긍정 감성 / 부정 감성 정도 의미하는 감성 지수(Polarity score) 가짐
- 감성지수는 단어위치, 주변 단어, 문맥, POS(Part of Speech) 참고해 결정됨
- 대표 패키지 - NLTK 패키지

NLTK - WordNet

- 시맨틱(semantic, 문맥상 의미) 정보 제공
- Synset : 단어가 가지는 문맥, 시맨틱 정보를 제공하는 WordNet의 핵심 개념
- 예측 성능은 그리 좋지 못함

8-5 감성 분석

대표적인 감성 사전

SentiWordNet

- WordNet의 Synset 개념을 감성 분석에 적용
- Synset별로 3가지 감성 점수(긍정, 부정, 객관성) 할당
- 객관성 지수 : 감성과 관계없이 얼마나 객관적인지 표현

VADER

- 소셜 미디어 텍스트 감성 분석 제공 패키지
- 뛰어난 감성 분석 결과, 비교적 빠른 수행시간 보장 > 대용량 텍스트 데이터에 잘 사용됨

Pattern

- 예측 성능면에서 가장 주목받는 패키지
- 파이썬 2.X에서만 동작

8-5 감성 분석 – 비지도 학습(SentiWordNet)

WordNet의 Synset

```
import nltk
nltk.download('all')
```

```
from nltk.corpus import wordnet as wn

term = 'present'

# 'present'라는 단어로 wordnet의 synsets 생성.
synsets = wn.synsets(term)
print('synsets() 반환 type:', type(synsets))
print('synsets() 반환 값 갯수:', len(synsets))
print('synsets() 반환 값:', synsets)
```

```
synsets() 반환 type : <class 'list'>
synsets() 반환 값 갯수: 18
synsets() 반환 값 : [Synset('present.n.01'), Synset('present.n.02'), Synset('present.n.03'), Synset('show.v.01'), S
```

synsets(지정단어)

- 반환 값: 여러 개의 synset 객체를 가지는 리스트
- Synset 객체의 파라미터 – POS 태그('의미.품사.인덱스') 예) Synset('present.n.01')

8-5 감성 분석 – 비지도 학습(SentiWordNet)

Synset 객체의 속성

```
for synset in synsets :  
    print('##### Synset name : ', synset.name(), '#####')  
    print('POS :', synset.lexname())  
    print('Definition:', synset.definition())  
    print('Lemmas:', synset.lemma_names())
```

- synset객체.name : POS 태그
- synset객체.lexname : 품사, POS
- synset객체.definition : 단어의 정의
- synset객체.lemma_names : 동의어 리스트

```
##### Synset name : present.n.01 #####  
POS : noun.time  
Definition: the period of time that is happening now; any continuous stretch of time including the moment of speech  
Lemmas: ['present', 'nowadays']  
##### Synset name : present.n.02 #####  
POS : noun.possession  
Definition: something presented as a gift  
Lemmas: ['present']  
##### Synset name : present.n.03 #####  
POS : noun.communication  
Definition: a verb tense that expresses actions or states at the time of speaking  
Lemmas: ['present', 'present_tense']  
##### Synset name : show.v.01 #####  
POS : verb.perception  
Definition: give an exhibition of to an interested audience  
Lemmas: ['show', 'demo', 'exhibit', 'present', 'demonstrate']  
##### Synset name : present.v.02 #####  
POS : verb.communication  
Definition: bring forward and present to the mind  
Lemmas: ['present', 'represent', 'lay_out']  
##### Synset name : stage.v.01 #####  
POS : verb.creation  
Definition: perform (a play), especially on a stage  
Lemmas: ['stage', 'present', 'represent']  
##### Synset name : present.v.04 #####  
POS : verb.possession  
Definition: hand over formally  
Lemmas: ['present', 'submit']  
##### Synset name : present.v.05 #####  
POS : verb.stative  
Definition: introduce  
Lemmas: ['present', 'pose']  
##### Synset name : award.v.01 #####  
POS : verb.possession  
Definition: give, especially as an honor or reward  
Lemmas: ['award', 'present']  
##### Synset name : give.v.08 #####  
POS : verb.possession  
Definition: give as a present; make a gift of  
Lemmas: ['give', 'gift', 'present']  
##### Synset name : deliver.v.01 #####  
POS : verb.communication  
Definition: deliver (a speech, oration, or idea)  
Lemmas: ['deliver', 'present']
```

8-5 감성 분석 – 비지도 학습(SentiWordNet)

Synset 객체의 메소드

synset객체.path_similarity()

- 단어간의 유사도 확인

```
# synset 객체를 단어별로 생성합니다.
tree = wn.synset('tree.n.01')
lion = wn.synset('lion.n.01')
tiger = wn.synset('tiger.n.02')
cat = wn.synset('cat.n.01')
dog = wn.synset('dog.n.01')

entities = [tree , lion , tiger , cat , dog]
similarities = []
entity_names = [ entity.name().split('.')[0] for entity in entities]

# 단어별 synset 들을 iteration 하면서 다른 단어들의 synset과 유사도를 측정합니다.
for entity in entities:
    similarity = [ round(entity.path_similarity(compared_entity), 2) for compared_entity in entities ]
    similarities.append(similarity)

# 개별 단어별 synset과 다른 단어의 synset과의 유사도를 DataFrame형태로 저장합니다.
similarity_df = pd.DataFrame(similarities , columns=entity_names,index=entity_names)
similarity_df
```

	tree	lion	tiger	cat	dog
tree	1.00	0.07	0.07	0.08	0.12
lion	0.07	1.00	0.33	0.25	0.17
tiger	0.07	0.33	1.00	0.25	0.17
cat	0.08	0.25	0.25	1.00	0.20
dog	0.12	0.17	0.17	0.20	1.00

8-5 감성 분석 실습 – 비지도 학습(SentiWordNet)

SentiWordNet의 Senti_Synset 클래스

```
import nltk
from nltk.corpus import sentiwordnet as swn

senti_synsets = list(swn.senti_synsets('slow'))
print('senti_synsets() 반환 type:', type(senti_synsets))
print('senti_synsets() 반환 값 갯수:', len(senti_synsets))
print('senti_synsets() 반환 값:', senti_synsets)
```

```
senti_synsets() 반환 type: <class 'list'>
senti_synsets() 반환 값 갯수: 11
senti_synsets() 반환 값: [SentiSynset('decelerate.v.01'), SentiSynset('slow.v.02'), SentiSynset('slow.v.03'), SentiSynse
```

swn.senti_synsets(지정단어)

- 반환 값: 여러 개의 SentiSynset 객체를 가지는 리스트
- Synset 객체와 유사

8-5 감성 분석 실습 – 비지도 학습(SentiWordNet)

Senti_Synset 클래스 감성지수, 객관성 지수

```
import nltk
from nltk.corpus import sentiwordnet as swn

father = swn.senti_synset('father.n.01')
print('father 긍정감성 지수: ', father.pos_score())
print('father 부정감성 지수: ', father.neg_score())
print('father 객관성 지수: ', father.obj_score())
print('\n')
fabulous = swn.senti_synset('fabulous.a.01')
print('fabulous 긍정감성 지수: ', fabulous.pos_score())
print('fabulous 부정감성 지수: ', fabulous.neg_score())
```

```
father 긍정감성 지수: 0.0
father 부정감성 지수: 0.0
father 객관성 지수: 1.0
```

```
fabulous 긍정감성 지수: 0.875
fabulous 부정감성 지수: 0.125
```

- 긍정 감성 지수 : SentiSynset객체.pos_score()
- 부정 감성 지수 : SentiSynset객체.neg_score()
- 객관성 지수 : SentiSynset객체.obj_score()

8-5 감성 분석 실습 – 비지도 학습(SentiWordNet)

SentiWordNet을 이용한 영화 감상평 감성 분석 과정

Step ①

- 문서를 문장 단위로 분해

Step ②

- 문장을 단어단위로 토큰화, 품사 태깅

Step ③

- 품사 태깅된 단어 기반으로 synset/senti_synset 객체 생성

Step ④

- senti_synset 객체 > 긍정,부정 감성지수 구하기 + 합산 > 임계치 기반 결정

8-5 감성 분석 실습 – 비지도 학습(SentiWordNet)

SentiWordNet을 이용한 영화 감상평 감성 분석 과정에 따라 함수 정의

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import sentiwordnet as swn
from nltk import sent_tokenize, word_tokenize, pos_tag
```

```
def swn_polarity(text):
    # 감성 지수 초기화
    sentiment = 0.0
    tokens_count = 0
```

```
    lemmatizer = WordNetLemmatizer()
    raw_sentences = sent_tokenize(text)
    # 분해된 문장별로 단어 토큰 -> 품사 태깅 후에 SentiSynset 생성 -> 감성 지수 합산
```

```
    for raw_sentence in raw_sentences:
        # NLTK 기반의 품사 태깅 문장 추출
        tagged_sentence = pos_tag(word_tokenize(raw_sentence))
        for word, tag in tagged_sentence:
            # Step 2 - 문장을 단어단위로 토큰화, 품사 태깅
```

```
            # WordNet 기반 품사 태깅과 어근 추출
            wn_tag = penn_to_wn(tag)
            if wn_tag not in (wn.NOUN, wn.ADJ, wn.ADV):
                continue
```

```
            lemma = lemmatizer.lemmatize(word, pos=wn_tag)
```

```
            if not lemma:
                continue
            # Step 3 - 품사 태깅된 단어 기반으로 synset/senti_synset 객체 생성
```

```
            # 어근을 추출한 단어와 WordNet 기반 품사 태깅을 입력해 Synset 객체를 생성.
```

```
            synsets = wn.synsets(lemma, pos=wn_tag)
```

```
            if not synsets:
                continue
```

```
            # sentiwordnet의 감성 단어 분석으로 감성 synset 추출
```

```
            # 모든 단어에 대해 긍정 감성 지수는 +로 부정 감성 지수는 -로 합산해 감성 지수 계산.
```

```
            synset = synsets[0]
```

```
            swn_synset = swn.senti_synset(synset.name())
```

```
            sentiment += (swn_synset.pos_score() - swn_synset.neg_score())
```

```
            tokens_count += 1
```

```
    if not tokens_count:
        return 0
```

Step 4 - senti_synset 객체 > 긍정,부정 감성지수 구하기 + 합산 > 임계치 기반 결정

```
    # 총 score가 0 이상일 경우 긍정(Positive) 1, 그렇지 않을 경우 부정(Negative) 0 반환
```

```
    if sentiment >= 0:
        return 1
```

```
    return 0
```

```
from nltk.corpus import wordnet as wn
```

```
# 간단한 NLTK PennTreebank Tag를 기반으로 WordNet기반의 품사 Tag로 변환
```

```
def penn_to_wn(tag):
    if tag.startswith('J'):
        return wn.ADJ
    elif tag.startswith('N'):
        return wn.NOUN
    elif tag.startswith('R'):
        return wn.ADV
    elif tag.startswith('V'):
        return wn.VERB
    return
```


8-5 감성 분석 실습 – 비지도 학습(SentiWordNet)

SentiWordNet 학습/예측/평가

```
review_df['preds'] = review_df['review'].apply( lambda x : swn_polarity(x) )
y_target = review_df['sentiment'].values
preds = review_df['preds'].values
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score
from sklearn.metrics import recall_score, f1_score, roc_auc_score
import numpy as np
```

```
print(confusion_matrix( y_target, preds))
print("정확도:", np.round(accuracy_score(y_target , preds), 4))
print("정밀도:", np.round(precision_score(y_target , preds),4))
print("재현율:", np.round(recall_score(y_target, preds), 4))
```

```
[[7669 4831]
 [3635 8865]]
정확도: 0.6614
정밀도: 0.6473
재현율: 0.7092
```

8-5 감성 분석 실습 – 비지도 학습(VADER)

VADER

- 소셜 미디어 텍스트 감성 분석 용도의 Lexicon
- SentimentIntensityAnalyzer 클래스
- NLTK의 서브 모듈/ 단독 패키지 둘다 지원
 - 단독 패키지 : `pip install vaderSentiment`,
`from vaderSentiment import SentimentIntensityAnalyzer`

8-5 감성 분석 실습 – 비지도 학습(VADER)

SentimentIntensityAnalyzer객체.polarity_scores()를 이용해 VADER 감성 분석 수행

```
[18] from nltk.sentiment.vader import SentimentIntensityAnalyzer

      senti_analyzer = SentimentIntensityAnalyzer()
      senti_scores = senti_analyzer.polarity_scores(review_df['review'][0])
      print(senti_scores)
```

↔ {'neg': 0.13, 'neu': 0.743, 'pos': 0.127, 'compound': -0.7943}

객체.polarity_scores(비정형 텍스트)

- 반환 : 딕셔너리 형태
- neg, pos : 부정, 긍정 감성 지수
- neu : 중립 감성 지수
- compound : 부정/긍정/중립 지수를 적절히 조합해 [-1,1]안의 지수로 표현
0.1 이상 = 긍정 감성

8-5 감성 분석 실습 – 비지도 학습(VADER)

VADER 감성 분석 학습/예측/평가

```
def vader_polarity(review, threshold=0.1): 긍정지수 임계값 = 0.1
    analyzer = SentimentIntensityAnalyzer()
    scores = analyzer.polarity_scores(review)

    # compound 값에 기반하여 threshold 입력값보다 크면 1, 그렇지 않으면 0을 반환
    agg_score = scores['compound']
    final_sentiment = 1 if agg_score >= threshold else 0
    return final_sentiment

# apply lambda 식을 이용하여 레코드별로 vader_polarity( )를 수행하고 결과를 'vader_preds'에 저장
review_df['vader_preds'] = review_df['review'].apply( lambda x : vader_polarity(x, 0.1) )
y_target = review_df['sentiment'].values
vader_preds = review_df['vader_preds'].values

print(confusion_matrix( y_target, vader_preds))
print("정확도:", np.round(accuracy_score(y_target , vader_preds),4))
print("정밀도:", np.round(precision_score(y_target , vader_preds),4))
print("재현율:", np.round(recall_score(y_target, vader_preds),4))
```

```
[[ 6747  5753]
 [ 1858 10642]]
정확도: 0.6956
정밀도: 0.6491
재현율: 0.8514
```

THANK YOU

